

SET-Semesterprojekt Sommersemester 2026

KI-gestützter Anamnese-Agent für die hausärztliche Praxis

Institute for Applied Artificial Intelligence and Robotics (A²IR)
Fakultät für Informationstechnik
Technische Hochschule Mannheim

Datum: 12. Mai 2026

Ort: Gebäude K, Raum K106, Paul-Wittsack-Straße 10, 68163 Mannheim

KI-gestützter Anamnese-Agent für die hausärztliche Praxis

1. Kurzbeschreibung

In diesem 7-wöchigen Semesterprojekt entwickeln interdisziplinäre Teams einen prototypischen KI-gestützten Anamnese-Agenten für eine hausärztliche Praxis. Der Agent unterstützt die strukturierte Erhebung relevanter Informationen in klar abgegrenzten hausärztlichen Szenarien, erkennt definierte Warnzeichen, bindet Vitalparameter und Gerätesimulatoren ein und erstellt eine strukturierte Zusammenfassung für ärztliches Personal.

Das System ist ausdrücklich ein **Ausbildungs-, Demonstrations- und Assistenzsystem**. Es stellt keine Diagnosen, gibt keine Therapieempfehlungen, nimmt keine Entwarnung vor und ersetzt keine ärztliche Bewertung. Die finale medizinische Einschätzung erfolgt ausschließlich durch ärztliches Personal.

Die Entwicklung erfolgt professionell und KI-gestützt. Bevorzugtes Werkzeug ist OpenAI Codex mit einem aktuellen leistungsfähigen Modell, zum Beispiel GPT-5.5. Andere KI-Entwicklungstools sind zulässig, wenn deren Einsatz nachvollziehbar dokumentiert, kritisch geprüft und reflektiert wird.

2. Lernziele

Nach erfolgreichem Abschluss des Projekts können die Studierenden:

- Anforderungen für ein medizinisch sensibles Softwaresystem erheben, priorisieren und nachvollziehbar dokumentieren.
- medizinische Szenarien in strukturierte Dialog-, Zustands- und Datenmodelle übersetzen.
- einen modularen Softwareprototyp in Python oder C++ mit Qt entwerfen, implementieren, testen und präsentieren.
- KI-Entwicklungstools wie Codex produktiv, kontrolliert und reflektiert einsetzen.
- KI-generierten Code hinsichtlich Korrektheit, Sicherheit, Testbarkeit, Wartbarkeit und fachlicher Angemessenheit überprüfen.
- Red-Flag-Logik, Eskalationspfade und Human-in-the-loop-Prinzipien technisch umsetzen.
- synthetische Patientendaten importieren, strukturieren und datensparsam verarbeiten.
- Geräteschnittstellen oder Gerätesimulatoren für Vitalparameter integrieren.

- automatisierte Tests für Kernlogik, Datenimport, Dialogsteuerung, Konfiguration und Gerätesimulation entwickeln.
- medizinische, ethische und rechtliche Grenzen eines KI-Assistenzsystems erklären.

3. Ausgangssituation und Motivation

Hausärztliche Praxen stehen unter hohem Zeitdruck. Wiederkehrende Beratungsanlässe wie Atemwegsinfekte, Brustschmerz, auffällige Blutdruckwerte oder metabolische Verlaufskontrollen erfordern eine strukturierte Erhebung von Beschwerden, Vorerkrankungen, Medikamenten, Risikofaktoren und Warnzeichen. Gleichzeitig darf eine technische Assistenz nicht den Eindruck erwecken, ärztliche Verantwortung zu übernehmen.

Das Projekt verbindet daher drei Perspektiven:

1. **Software Engineering:** professionelle Entwicklung eines modularen, testbaren Prototyps.
2. **Technische Informatik:** Einbindung von Schnittstellen, Gerätesimulatoren, Konfiguration, Logging und Benutzeroberfläche.
3. **Medizintechnik:** strukturierte Erhebung von Gesundheitsinformationen, Vitalparametern, Sicherheitsgrenzen und medizinischer Übergabe.

Die Studierenden sollen erleben, dass KI-gestützte Softwareentwicklung nicht bedeutet, unkontrolliert Code generieren zu lassen. Vielmehr müssen Anforderungen, Tests, Architektur, Reviews, Dokumentation und fachliche Validierung besonders sorgfältig durchgeführt werden.

4. Fachlicher Hintergrund

Das Projekt orientiert sich an aktuellen hausärztlichen und versorgungsnahen Leitlinien, ohne diese vollständig in Software zu operationalisieren. Geeignete fachliche Bezugspunkte sind insbesondere hausärztliche Leitlinien zu Brustschmerz, Husten, Hypertonie und Typ-2-Diabetes, zum Beispiel Leitlinien und Patienteninformationen von AWMF, DEGAM und Nationalen VersorgungsLeitlinien.

Für Brustschmerz ist insbesondere eine strukturierte Erfassung von Symptomen, Risikofaktoren und Warnzeichen relevant. Didaktisch kann der Marburger Herzscore als Beispiel für eine hausärztliche Entscheidungsregel betrachtet werden. Eine Softwareberechnung darf jedoch nicht als Diagnose, Entwarnung oder finale Risikobewertung ausgegeben werden.

Für Husten und Atemwegsbeschwerden stehen Symptombdauer, Fieber, Atemnot, Auswurf, thorakale Schmerzen, Vorerkrankungen und Warnzeichen im Vordergrund. Bei Hypertonie und Typ-2-Diabetes sind strukturierte Vitalparameter, Verlauf, Risikofaktoren, Medikamente, Adhärenz und Warnzeichen relevant.

Die Studierenden müssen in ihrer Dokumentation angeben, welche Leitlinien, Patienteninformationen oder fachlichen Quellen sie verwendet haben. Die Quellen sind nicht blind in Code umzusetzen. Aus medizinischen Vorgaben werden im Projekt ausschließlich didaktisch geeignete Anforderungen an Anamnese, Warnhinweise, strukturierte Dokumentation und Eskalation abgeleitet.

5. Eingrenzung auf vier hausärztliche Szenarien

Die Mindestanforderung umfasst mindestens drei vollständig implementierte Szenarien. Die Projektbeschreibung gibt jedoch vier Standardszenarien vor, aus denen die Teams auswählen und priorisieren. Leistungsstarke Teams sollen alle vier Szenarien vollständig umsetzen.

5.1 Szenario A: Akuter Husten / Atemwegsinfekt / Verdacht auf Pneumonie

Zu erfassen sind mindestens:

- Symptombdauer,
- Hustenart,
- Fieber oder Schüttelfrost,
- Dyspnoe oder Belastungsdyspnoe,
- Auswurf, Farbe und Blutbeimengung,
- thorakale Schmerzen,
- Vorerkrankungen, insbesondere Asthma, COPD, Herzinsuffizienz oder Immunsuppression,
- relevante Risikofaktoren,
- aktuelle Medikamente.

Der Agent muss Warnzeichen erkennen, insbesondere Atemnot, Hämoptysen, hohes Fieber, reduzierten Allgemeinzustand, Verwirrtheit, Zyanose, relevante Vorerkrankungen oder rasche Verschlechterung.

Der Pulsoximeter wird **nicht routinemäßig** eingesetzt, sondern nur bei medizinisch begründetem Bedarf, zum Beispiel bei Atemnot, Verdacht auf relevante respiratorische Einschränkung oder definierter Red-Flag-Konstellation. Die Auslösebedingungen sind im Projektbericht zu begründen.

5.2 Szenario B: Brustschmerz in der Hausarztpraxis

Zu erfassen sind mindestens:

- Lokalisation,
- Beginn und Dauer,
- Schmerzcharakter,
- Ausstrahlung,
- Belastungsabhängigkeit,
- Begleitsymptome wie Atemnot, Übelkeit, Kaltschweißigkeit, Synkope oder ausgeprägte Schwäche,
- kardiovaskuläre Risikofaktoren,
- bekannte Vorerkrankungen,
- aktuelle Medikamente.

Dieses Szenario verlangt eine besonders konsequente Notfall- und Red-Flag-Erkennung. Der Agent darf keine Entwarnung aussprechen. Eine strukturierte Risikoeinschätzung kann didaktisch mit Bezug auf hausärztliche Entscheidungsregeln wie den Marburger Herzscore vorbereitet werden. Die Score-Berechnung darf jedoch nicht als Diagnose oder finale Risikobewertung dargestellt werden. Jede kritische Konstellation führt zu einem klaren Eskalationshinweis an ärztliches Personal.

5.3 Szenario C: Hypertonie-Kontrolle / auffälliger Blutdruckwert

Zu erfassen sind mindestens:

- Blutdruckwerte mit systolischem und diastolischem Wert,
- Puls, sofern verfügbar,
- Messbedingungen,
- Beschwerden wie Kopfschmerz, Brustschmerz, Atemnot, neurologische Symptome oder Sehstörungen,
- bekannte Vorerkrankungen,
- Medikamente und Adhärenz,
- Lebensstilfaktoren,
- Risikofaktoren,
- Gewicht.

Eine Bluetooth-Blutdruckmanschette oder ein Gerätesimulator ist einzubinden. Gewicht soll über Bluetooth-Waage, Gerätesimulator oder mündliche Eingabe erfasst werden. Der Agent erkennt mögliche hypertensive Warnzeichen und leitet in solchen Fällen an ärztliches Personal weiter.

5.4 Szenario D: Typ-2-Diabetes / metabolische Verlaufskontrolle

Zu erfassen sind mindestens:

- bekannte Diagnosen,
- aktuelle Medikamente,
- Hinweise auf Hypoglykämie oder Hyperglykämie,
- Gewichtsverlauf,
- Blutdruck,
- Lebensstilfaktoren,
- Folgeerkrankungen oder bekannte Komplikationen,
- Kontrolltermine und relevante Vorbefunde, soweit synthetisch vorgegeben,
- offene Fragen der Patientin oder des Patienten.

Gewicht und Blutdruck werden als strukturierte Verlaufsparemeter genutzt. Das System erstellt eine Gesprächs- und Befundvorlage für ärztliches Personal, gibt aber keine Therapieempfehlung, keine Dosisanpassung und keine medizinische Entwarnung.

5.5 Optionales fünftes Szenario

Als optionale Erweiterung kann ein fünftes Szenario umgesetzt werden, zum Beispiel:

- nicht-spezifischer Kreuzschmerz mit Red-Flag-Erkennung,
- Harnwegsbeschwerden,
- Medikations-Check vor Hausarzttermin,
- Impfstatus-Vorbereitung anhand synthetischer Daten.

Das fünfte Szenario ist nicht Bestandteil der Mindestanforderungen und darf erst begonnen werden, wenn Kernfunktionalität, Tests, Dokumentation und Sicherheitsgrenzen stabil umgesetzt sind.

6. Aufgabenstellung

Entwickeln Sie als Team einen lauffähigen Prototyp eines KI-gestützten Anamnese-Agenten für eine hausärztliche Praxis. Der Agent soll eine Tagesliste synthetischer Patientinnen und Patienten über eine URL laden, eine Person identifizieren, ein medizinisches Szenario auswählen, eine strukturierte assistierte Anamnese durchführen, Vitalparameter erfassen oder abfragen, Red Flags erkennen und eine strukturierte Zusammenfassung für ärztliches Personal erzeugen.

Der Kernprozess umfasst verbindlich:

1. Laden der Tagesliste über eine URL.
2. Auswahl des aktuellen Patienten.
3. Identitätsprüfung, mindestens über Name und Geburtsdatum.
4. Auswahl des Szenarios.
5. Erklärung der Rolle des Systems.
6. Einholung einer einfachen Zustimmung zur assistierten Anamnese.
7. Durchführung der strukturierten Anamnese.
8. Erfassung oder Abfrage von Vitalparametern.
9. Erkennung von Red Flags und Eskalation an ärztliches Personal.
10. Erstellung einer strukturierten Zusammenfassung.
11. Übergabe an ärztliches Personal.
12. Abschluss des Dialogs.

7. Funktionale Anforderungen

7.1 Priorisierung

Die Anforderungen werden in drei Stufen eingeteilt:

Muss-Anforderungen: erforderlich für das Bestehen.

Soll-Anforderungen: erwartet für eine gute bis sehr gute Leistung.

Kann-Anforderungen: optionale Erweiterungen für leistungsstarke Teams.

7.2 Patientenlisten-Import

Muss:

- Import einer JSON-Liste über eine konfigurierbare URL.
- Verarbeitung synthetischer Patientendaten.
- Validierung der Mindestfelder.
- verständliche Fehlermeldung bei nicht erreichbarer URL, ungültigem JSON oder fehlenden Pflichtfeldern.

Soll:

- Schema-Validierung,
- Sortierung nach Terminzeit,
- Filterung nach geplantem Beratungsanlass,
- robuste Behandlung optionaler Felder.

Beispielhaftes JSON-Format:

```
[
  {
    "patient_id": "P-2026-0001",
    "name": {
      "given": "Anna",
      "family": "Muster"
    },
    "date_of_birth": "1972-04-18",
    "appointment_time": "2026-05-12T09:15:00+02:00",
    "planned_reason": "Hypertonie-Kontrolle",
    "preferred_interaction": "keyboard",
    "synthetic": true
  },
  {
    "patient_id": "P-2026-0002",
    "name": {
      "given": "Mehmet",
      "family": "Yilmaz"
    },
    "date_of_birth": "1961-11-03",
    "appointment_time": "2026-05-12T09:45:00+02:00",
    "planned_reason": "Brustschmerz",
    "preferred_interaction": "voice",
    "synthetic": true
  }
]
```

7.3 Patientenidentifikation

Muss:

- Auswahl eines Patienten aus der Tagesliste.

- Abfrage von Name und Geburtsdatum.
- Abbruch oder erneute Prüfung bei Nichtübereinstimmung.

Soll:

- Hinweis an Praxispersonal bei wiederholter Fehlidentifikation.
- Logging des Identifikationsstatus ohne unnötige Speicherung sensibler Dialogdetails.

7.4 Szenarioauswahl**Muss:**

- Auswahl aus den vier vorgegebenen Standardszenarien.
- Zuordnung anhand des geplanten Beratungsanlasses oder manueller Auswahl.
- Anzeige, welches Szenario aktiv ist.

Soll:

- Plausibilitätsprüfung, wenn Beratungsanlass und ausgewähltes Szenario nicht zusammenpassen.

7.5 Rollenerklärung und Zustimmung**Muss:**

- verständliche Erklärung, dass das System ein Assistenzsystem ist,
- Hinweis, dass keine Diagnose und keine Therapieempfehlung erfolgt,
- einfache Zustimmung zur Durchführung der assistierten Anamnese,
- Möglichkeit zum Abbruch.

7.6 Dialogsteuerung**Muss:**

- strukturierter Dialog mit definierten Zuständen,
- Erfassung aller Pflichtinformationen je Szenario,
- Rückfragen bei unklaren oder fehlenden Angaben,
- sauberer Abschluss des Dialogs.

Soll:

- kontextabhängige Folgefragen,
- Zusammenfassung vor Übergabe,
- Trennung zwischen freitextlichen Angaben und strukturierten Feldern.

7.7 Vitalparameter und Geräte

Muss:

- mindestens ein Gerätesimulator oder eine echte Geräteschnittstelle,
- konfigurierbare Auswahl zwischen automatischer Messwertübernahme und manueller Eingabe,
- Validierung plausibler Wertebereiche,
- Kennzeichnung, ob ein Wert gemessen, simuliert oder mündlich angegeben wurde.

Soll:

- Simulatoren für Blutdruck, Gewicht und Pulsoximetrie,
- reproduzierbare Testdaten,
- Fehlerfälle wie Verbindungsabbruch oder ungültiger Messwert.

7.8 Red-Flag-Erkennung

Muss:

- regelbasierte Erkennung definierter Warnzeichen je Szenario,
- unmittelbarer Eskalationshinweis bei kritischer Konstellation,
- keine Fortsetzung als Routineanamnese ohne Warnhinweis,
- Protokollierung der ausgelösten Red Flag.

Soll:

- nachvollziehbare Begründung, welche Eingabe welche Red Flag ausgelöst hat,
- unterscheidbare Eskalationsstufen, zum Beispiel *ärztliche Prüfung erforderlich* und *sofortige ärztliche Übernahme erforderlich*.

7.9 Strukturierte Zusammenfassung

Muss:

- Ausgabe einer strukturierten Zusammenfassung für ärztliches Personal,
- Trennung von Patientenantworten, Messwerten, Red Flags und offenen Punkten,
- Export als JSON oder Markdown.

Soll:

- zusätzlicher PDF- oder HTML-Export,
- maschinenlesbare Struktur für spätere FHIR-Erweiterung,
- klare Markierung synthetischer Daten.

7.10 Interaktionsformen

Muss:

- Bildschirmausgabe mit Tastatureingabe.

Soll:

- Sprachinteraktion für Ein- oder Ausgabe,
- stabiler Fallback auf Tastatureingabe.

Geeignete Bibliotheken oder Dienste für Spracheingabe und Sprachausgabe dürfen verwendet werden. Wenn Spracheingabe technisch nicht stabil realisierbar ist, muss die Tastaturinteraktion vollständig funktionsfähig bleiben.

7.11 Grafische Ausgabe und Avatar

Muss:

- grafische Ausgabe,
- einfacher abstrakter Avatar,
- mindestens drei unterscheidbare Gesprächszustände.

Soll:

- Zustände für Begrüßen, Zuhören, Nachdenken, Rückfrage, Messung, Warnhinweis und Abschluss,
- einfache Animation oder Zustandswechsel.

Die Umsetzung ist frei wählbar, zum Beispiel Qt-Grafik, Web-Frontend, GIF-Animation, Canvas, SVG oder einfache Echtzeit-Animation.

8. Nichtfunktionale Anforderungen

- **Nachvollziehbarkeit:** Red Flags, Konfigurationsentscheidungen und Ausgaben müssen nachvollziehbar sein.
- **Testbarkeit:** medizinische Logik, Dialogzustände, Import und Gerätesimulatoren müssen ohne UI automatisiert testbar sein.
- **Modularität:** UI, Dialoglogik, Red-Flag-Regeln, Gerätezugriff und Datenimport sind zu trennen.
- **Robustheit:** ungültige Eingaben, fehlende Messwerte, Gerätefehler und Abbrüche müssen kontrolliert behandelt werden.
- **Benutzbarkeit:** Interaktion soll klar, freundlich, langsam genug und ohne Fachjargon erfolgen.
- **Wartbarkeit:** Regeln, Szenarien und Geräteoptionen sollen konfigurierbar oder leicht erweiterbar sein.
- **Datensparsamkeit:** Es werden nur Daten erhoben, die für das jeweilige Szenario und die Demonstration erforderlich sind.

- **Sicherheit:** Das System darf keine medizinisch riskanten Aussagen, Entwarnungen oder Therapiehinweise generieren.
- **Reproduzierbarkeit:** Demo-Szenarien, Testdaten und Konfigurationen müssen reproduzierbar sein.

9. Medizinische, ethische und rechtliche Grenzen

Das Projekt behandelt Gesundheitsdaten ausschließlich anhand synthetischer Patientendaten. Die Grundprinzipien einer DSGVO-orientierten Umsetzung sind Rechtmäßigkeit, Transparenz, Zweckbindung, Datenminimierung, Speicherbegrenzung, Integrität, Vertraulichkeit und Rechenschaftspflicht. Daraus ergeben sich für das Projekt folgende verbindliche Grenzen:

- Es werden keine realen Patientendaten verwendet.
- Testdaten und hypothetische Praxisdaten sind strikt zu trennen.
- Sensible Daten werden nicht dauerhaft gespeichert, sofern kein begründetes Datenschutzkonzept vorliegt.
- Exportdateien enthalten nur die für die Demo notwendigen Informationen.
- Das System stellt keine Diagnose.
- Das System trifft keine Therapieentscheidung.
- Das System gibt keine Aussage wie “Sie haben nichts Ernstes” aus.
- Das System ersetzt keine ärztliche Beratung.
- Das System ist jederzeit als Assistenzsystem kenntlich.
- Finale medizinische Bewertung und Entscheidung bleiben beim ärztlichen Personal.
- Kritische Entscheidungen und Red-Flag-Auslösungen werden protokolliert.
- Abbrüche, Notfallsymptome und technische Fehler werden sicher behandelt.
- Bei Red Flags erfolgt eine klare Eskalation an medizinisches Personal.
- Grenzen des KI-Systems werden für Nutzende transparent dargestellt.

Der Prototyp ist kein Medizinprodukt, nicht CE-gekennzeichnet und nicht für den klinischen Einsatz oder die eigenständige Nutzung in einer realen Versorgungssituation bestimmt. Eine reale Praxisintegration würde eine gesonderte Prüfung nach Datenschutzrecht, Medizinprodukterecht, ärztlichem Berufsrecht und gegebenenfalls AI Act erfordern. Insbesondere müssten Zweckbestimmung, Risikoklasse, Verantwortlichkeiten, klinische Bewertung, technische Sicherheit, Qualitätsmanagement, Dokumentation und Überwachung nach Inbetriebnahme separat betrachtet werden.

10. Technische Rahmenbedingungen

Die Umsetzung erfolgt wahlweise mit:

- Python,
- oder C++ mit Qt,

- optional LangGraph,
- alternativ direkte Programmierung ohne Agentenframework.

Eine Laufzeit-KI ist nicht verpflichtend. Ein regelbasierter Dialogagent mit expliziter Zustandsmaschine erfüllt die Mindestanforderungen, sofern Dialogführung, Red-Flag-Erkennung, Zusammenfassung und Sicherheitsgrenzen nachvollziehbar umgesetzt und getestet sind. Wird ein LLM zur Laufzeit eingesetzt, dürfen medizinisch sicherheitsrelevante Aussagen nicht allein durch das LLM erzeugt oder bewertet werden. Red Flags, Eskalationsentscheidungen und sicherheitsrelevante Zusammenfassungen müssen durch deterministische Regeln, Tests und technische Sicherheitsgrenzen kontrolliert werden.

10.1 Python-Variante

Geeignet für Teams, die schnell Prototypen entwickeln, Tests automatisieren und Geräte- oder API-Simulatoren integrieren möchten. Mögliche Bausteine sind:

- FastAPI oder Flask für lokale Services,
- PyQt, Tkinter, NiceGUI, Streamlit oder Web-Frontend für UI,
- pytest für Tests,
- pydantic oder jsonschema für Datenvalidierung,
- Logging-Modul für Protokollierung.

10.2 C++-Variante mit Qt

Geeignet für Teams mit Fokus auf Desktop-UI, Zustandsautomaten und hardwarenahe Schnittstellen. Mögliche Bausteine sind:

- Qt/QML,
- Qt Bluetooth oder simulierte Geräteadapter,
- JSON-Verarbeitung mit Qt oder geeigneter Bibliothek,
- Catch2 oder GoogleTest,
- strukturierte Logging-Komponente.

10.3 LangGraph als optionale Architekturvariante

LangGraph ist eine mögliche, aber nicht verpflichtende Architekturvariante. Für dieses Projekt ist LangGraph besonders dann sinnvoll, wenn ein Team den Dialog als expliziten Graphen mit Zuständen wie *Identifikation*, *Zustimmung*, *Anamnese*, *Messung*, *Red-Flag-Prüfung*, *Zusammenfassung* und *Abschluss* modellieren möchte.

Nicht sinnvoll ist LangGraph, wenn es nur eingesetzt wird, um technische Komplexität zu erhöhen. Eine direkte Zustandsmaschine ist für viele Teams ausreichend und kann didaktisch sogar klarer sein.

11. Empfohlene Systemarchitektur

Die Architektur soll modular aufgebaut sein. Eine sinnvolle Zielstruktur ist:

```
app/  
  main.py oder main.cpp  
  config/  
    settings.yaml  
    scenarios.yaml  
  patient_import/  
    patient_list_client  
    patient_schema  
  identity/  
    identity_check  
  dialogue/  
    dialogue_controller  
    state_machine  
    consent_flow  
  scenarios/  
    cough_scenario  
    chest_pain_scenario  
    hypertension_scenario  
    diabetes_scenario  
  medical_rules/  
    red_flag_engine  
    risk_rule_models  
  devices/  
    device_interface  
    blood_pressure_adapter  
    scale_adapter  
    pulse_oximeter_adapter  
    simulators  
  ui/  
    main_window oder web_frontend  
    avatar_component  
  output/  
    summary_builder  
    export_json  
    export_markdown  
  logging/  
    audit_logger  
  tests/  
    unit/  
    integration/  
    synthetic_cases/
```

11.1 Kernkomponenten

Patientenlisten-Import: lädt und validiert die Tagesliste.

Patientenidentifikation: prüft Name und Geburtsdatum.

Szenarioauswahl: wählt das passende Szenario manuell oder anhand des Beratungsanlasses.

Dialogsteuerung: verwaltet Zustände, Fragen, Antworten, Rückfragen und Abbruch.

Medizinische Anamnese-Logik: definiert Pflichtfragen und strukturierte Felder je Szenario.

Red-Flag-Erkennung: prüft regelbasiert Warnzeichen und Eskalationsbedingungen.

Geräteschnittstellen oder Simulatoren: liefern Messwerte für Blutdruck, Gewicht und Puls-oximetrie.

Konfigurationsmanagement: steuert Gerätebetrieb, Interaktionsform, Szenarien und Exportoptionen.

Strukturierte Ergebnisablage: erzeugt JSON, Markdown oder HTML.

Benutzeroberfläche: stellt Dialog, Eingaben, Status und Ergebnis dar.

Avatar-Komponente: visualisiert Gesprächszustände.

Test- und Logging-Komponente: ermöglicht Qualitätssicherung und Nachvollziehbarkeit.

11.2 Beispiel für Dialogzustände

START

```
-> LOAD_PATIENT_LIST
-> SELECT_PATIENT
-> VERIFY_IDENTITY
-> SELECT_SCENARIO
-> EXPLAIN_ROLE
-> REQUEST_CONSENT
-> STRUCTURED_HISTORY
-> VITAL_PARAMETERS
-> RED_FLAG_CHECK
-> ESCALATE_IF_NEEDED
-> BUILD_SUMMARY
-> HANDOVER
-> END
```

12. Vorschlag für den KI-gestützten Entwicklungsprozess

Der Entwicklungsprozess kombiniert klassische Softwareentwicklung mit kontrollierter KI-Unterstützung.

12.1 Grundprinzip

KI-Tools dürfen Code, Tests, Dokumentationsentwürfe, Refactoring-Vorschläge und Architekturvarianten erzeugen. Die Verantwortung bleibt jedoch beim Team. Jede KI-Ausgabe ist ein Vorschlag, kein geprüfter Befund.

12.2 Empfohlener Ablauf

1. Anforderungen in kleine, überprüfbare User Stories zerlegen.
2. Akzeptanzkriterien vor der Implementierung formulieren.
3. Für jede Story zuerst Testfälle oder Testideen definieren.
4. Codex mit kleinem, präzisem Auftrag einsetzen.

5. KI-generierten Code lokal prüfen, lesen, testen und refaktorisieren.
6. Code Review im Team durchführen.
7. Testergebnisse dokumentieren.
8. Reflexion des KI-Einsatzes fortlaufend notieren.

12.3 Codex-taugliche Arbeitspakete

Ein geeignetes Arbeitspaket für Codex ist:

- fachlich klein,
- technisch klar abgegrenzt,
- durch Tests überprüfbar,
- unabhängig von sensiblen Daten,
- mit erwarteter Ein- und Ausgabe beschrieben.

Gutes Beispiel:

Implementiere in Python eine Funktion `load_patient_list(url: str) -> list[Patient]`, die eine JSON-Liste lädt, Pflichtfelder validiert und bei Fehlern definierte Exceptions wirft. Schreibe pytest-Tests für gültiges JSON, fehlendes Geburtsdatum, ungültige URL und optionales Feld `planned_reason`. Verwende keine realen Patientendaten.

Schlechtes Beispiel:

Baue mir den ganzen medizinischen Agenten.

12.4 Prompt Engineering für Codex

Prompts sollen enthalten:

- Ziel der Aufgabe,
- betroffene Dateien,
- gewünschte Programmiersprache,
- Schnittstellen,
- Akzeptanzkriterien,
- Testanforderungen,
- medizinische und datenschutzbezogene Grenzen,
- explizite Verbote, zum Beispiel keine Diagnoseausgabe.

12.5 Prüfung KI-generierten Codes

Jeder KI-generierte Code muss mindestens geprüft werden auf:

- fachliche Angemessenheit,
- korrekte Behandlung von Fehlerfällen,
- Testbarkeit,
- Lesbarkeit,
- Sicherheitsgrenzen,
- Datenschutz,
- Abhängigkeiten und Lizenzen,
- keine verdeckte Nutzung realer Daten,
- keine medizinischen Aussagen außerhalb der Anforderungen.

12.6 Umgang mit Halluzinationen

Typische Risiken sind erfundene Bibliotheken, nicht existente APIs, falsche medizinische Regeln, unsichere Default-Werte und scheinbar plausible, aber ungetestete Logik. Das Team muss solche Vorschläge durch Dokumentation, Tests, Quellenprüfung und Code Review erkennen. Medizinische Regeln dürfen nicht allein aus KI-Ausgaben übernommen werden.

13. 7-Wochen-Zeitplan mit Meilensteinen

Woche	Schwerpunkt	Erwartete Ergebnisse	Abgabe/Meilenstein
1	Projektverständnis, Szenarien, Anforderungen, Risiken, Datenmodell	Teamrollen, initiale User Stories, Muss/Soll/Kann-Priorisierung, synthetisches Datenmodell, erste Risikoanalyse, Quellenliste	Kurzkonzept mit Anforderungen, Datenmodell und Risikoliste
2	Architektur, Dialogmodell, JSON-Import, erste UI-Skizze	Architekturdiagramm, Zustandsmodell, JSON-Import-Prototyp, Identitätsprüfung, UI-Wireframe, Testgrundgerüst	Architektur- und Dialogentwurf, lauffähiger Importtest
3	Kernlogik und erstes Szenario	Dialogsteuerung, erstes vollständig testbares Szenario, strukturierte Zusammenfassung, erste Red-Flag-Regeln	Zwischen-Demo 1: ein End-to-End-Szenario
4	Erweiterung auf weitere Szenarien, Gerätesimulation, Konfiguration	zweites und drittes Szenario, Gerätesimulator, Konfigurationsdatei, Fehlerfälle für Geräte und Import	Szenario- und Geräte-meilenstein
5	Avatar, UI, Red-Flag-Logik, strukturierte Abgabe	grafische Oberfläche, Avatarzustände, Red-Flag-Eskalation, Export als JSON oder Markdown, Logging kritischer Ereignisse	Zwischen-Demo 2: integrierter Prototyp
6	Tests, Integration, Dokumentation, Usability-Feedback	automatisierte Tests, Integrationstests, synthetische Testfälle, manuelle Usability-Rückmeldung, Dokumentationsentwurf	Testbericht, vorläufige Dokumentation
7	Stabilisierung, Abschlussdemo, Abgabe, Reflexion	stabiler Prototyp, Installationsanleitung, finale Dokumentation, Reflexion KI-Einsatz, Abschlusspräsentation	Abschlussdemo, Repository, Dokumentation, Reflexionsbericht

14. Erwartete Artefakte und Abgaben

Am Projektende sind abzugeben:

- lauffähiger Prototyp,
- Quellcode in einem nachvollziehbaren Repository,
- synthetische Beispiel-Patientenlisten,
- Konfigurationsdateien,
- automatisierte Tests,
- Testbericht mit Testergebnissen,
- technische und fachliche Dokumentation,
- Installations- und Bedienungsanleitung,

- Abschlusspräsentation,
- Demo-Skript,
- Reflexion des KI-Tool-Einsatzes,
- kurze Teamreflexion zu Zusammenarbeit und Projektmanagement.

15. Test- und Qualitätssicherungskonzept

15.1 Teststrategie

Die Tests sollen risikoorientiert priorisiert werden. Besonders kritisch sind medizinische Entscheidungs- und Dialoglogik, Red-Flag-Erkennung, Datenimport, Konfiguration und Gerätesimulation.

15.2 Automatisierte Tests

Verbindlich sind:

- Unit-Tests für Red-Flag-Regeln,
- Unit-Tests für Dialogzustände,
- Tests für JSON-Import und Schemafehler,
- Tests für Konfigurationsoptionen,
- Tests für Gerätesimulatoren,
- Tests für strukturierte Zusammenfassungen,
- Tests für Fehlerfälle und Abbrüche,
- Tests für Datenschutz- und Sicherheitsgrenzen, zum Beispiel keine Diagnoseausgabe.

15.3 Integrationstests

Mindestens drei synthetische End-to-End-Fälle sind umzusetzen:

1. unkritischer Routinefall ohne Red Flag,
2. Fall mit Red Flag und Eskalationshinweis,
3. Fall mit Geräte- oder Eingabefehler.

15.4 Manuelle Usability-Tests

Mindestens zwei Personen außerhalb des Entwicklungsteams sollen einen Demo-Dialog mit synthetischen Daten durchführen. Zu dokumentieren sind:

- Verständlichkeit der Fragen,
- Klarheit der Rolle des Systems,
- Abbruchmöglichkeit,
- Verständlichkeit von Warnhinweisen,
- UI- und Avatar-Wirkung,
- beobachtete Probleme.

15.5 Ziel-Codeabdeckung

Die folgenden Zielwerte sind realistisch und didaktisch angemessen:

Bereich	Mindestziel	Begründung
Medizinische Entscheidungs- und Dialoglogik	80 %	Höchstes Risiko; Regeln und Zustände müssen zuverlässig geprüft werden.
Datenimport und Konfiguration	75 %	Fehlerhafte Eingangsdaten sind wahrscheinlich und müssen robust behandelt werden.
Gerätesimulatoren	70 %	Simulatoren sollen reproduzierbar sein; echte Hardware kann ergänzend manuell getestet werden.
Strukturierte Ausgabe	70 %	Übergabedokumente müssen vollständig und konsistent sein.
UI- und Avatar-Code	30–50 % automatisiert plus manuelle Tests	UI-Verhalten ist teilweise schwer sinnvoll automatisiert zu testen; entscheidend sind Smoke-Tests und Usability-Prüfung.

Eine hohe Gesamt-Codeabdeckung ersetzt nicht die fachliche Prüfung. Entscheidend ist, dass sicherheitsrelevante Logik gezielt getestet wird.

16. Dokumentationsanforderungen

Die Dokumentation ist während des Projekts fortzuschreiben und muss mindestens enthalten:

- Projektziel,
- Anforderungen mit Muss/Soll/Kann-Priorisierung,
- Abgrenzung: Was leistet das System nicht?,
- Beschreibung der vier Szenarien,
- medizinischer und rechtlicher Kontext,
- verwendete Leitlinien oder fachliche Quellen,
- Systemarchitektur,
- Datenmodell,
- Dialogmodell,
- Red-Flag-Konzept,
- Begründung der Pulsoximeter-Auslösebedingungen,
- Konfigurationskonzept,
- Geräteschnittstellen oder Gerätesimulation,
- Testkonzept,

- Testergebnisse,
- Beschreibung des KI-Tool-Einsatzes,
- Beispiel-Prompts für Codex oder andere KI-Tools,
- Reflexion: hilfreiche, fehlerhafte oder riskante KI-Vorschläge,
- Installations- und Bedienungsanleitung,
- bekannte Einschränkungen,
- Ausblick auf eine mögliche Praxisintegration.

17. Bewertungsschema mit 100 Punkten

17.1 Punkteverteilung

Kategorie	Punkte
Anforderungsanalyse und fachliches Verständnis	8
Szenarienmodellierung und medizinische Angemessenheit	10
Softwarearchitektur	8
Implementierung und Funktionsumfang	12
KI-Agenten- und Dialogkonzept	8
Red-Flag-Erkennung und Sicherheitsabgrenzung	10
Datenschutz und ethische Reflexion	7
Testautomatisierung und Softwarequalität	10
Benutzeroberfläche und Avatar-Interaktion	6
Einsatz und Reflexion von KI-Entwicklungstools	7
Dokumentation	7
Präsentation und Demo	4
Teamarbeit und Projektmanagement	3
Summe	100

17.2 Bewertungskriterien je Kategorie

Anforderungsanalyse und fachliches Verständnis – 8 Punkte

Sehr gut: Anforderungen vollständig, priorisiert, testbar und fachlich sauber abgegrenzt.

Gut: Anforderungen überwiegend vollständig und nachvollziehbar, kleinere Lücken.

Ausreichend: zentrale Anforderungen vorhanden, aber unvollständig oder wenig testbar.

Mangelhaft: Anforderungen fehlen weitgehend oder widersprechen der Aufgabenstellung.

Szenarienmodellierung und medizinische Angemessenheit – 10 Punkte

Sehr gut: Szenarien sind strukturiert, realistisch, quellenbasiert und ohne unzulässige Diagnose- oder Therapieaussagen modelliert.

Gut: Szenarien sind plausibel und weitgehend vollständig, mit kleineren fachlichen Unschärfen.

Ausreichend: Szenarien funktionieren grundsätzlich, bleiben aber oberflächlich oder teilweise unsystematisch.

Mangelhaft: Szenarien sind medizinisch unangemessen, gefährlich oder nicht nachvollziehbar.

Softwarearchitektur – 8 Punkte

Sehr gut: klare Modularisierung, saubere Schnittstellen, gute Testbarkeit, Erweiterbarkeit und nachvollziehbare Architekturentscheidungen.

Gut: sinnvolle Architektur mit wenigen Kopplungs- oder Strukturproblemen.

Ausreichend: lauffähige, aber stark gekoppelte oder schwer erweiterbare Struktur.

Mangelhaft: keine erkennbare Architektur oder wesentliche Komponenten fehlen.

Implementierung und Funktionsumfang – 12 Punkte

Sehr gut: stabiler Prototyp mit mindestens vier gut integrierten Szenarien, Geräte- oder Simulatorlogik, Export und Fehlerbehandlung.

Gut: stabiler Prototyp mit mindestens drei vollständigen Szenarien und überwiegend sauberer Integration.

Ausreichend: lauffähiger Prototyp mit Mindestfunktionen, aber deutlichen Einschränkungen.

Mangelhaft: Prototyp nicht lauffähig oder zentrale Muss-Funktionen fehlen.

KI-Agenten- und Dialogkonzept – 8 Punkte

Sehr gut: klarer Zustands- oder Graphenentwurf, konsistente Dialogführung, gute Rückfragen und verständliche Patientensprache.

Gut: strukturierter Dialog mit kleineren Brüchen oder begrenzter Flexibilität.

Ausreichend: Dialog grundsätzlich nutzbar, aber starr, lückenhaft oder wenig robust.

Mangelhaft: Dialog unstrukturiert, verwirrend oder medizinisch riskant.

Red-Flag-Erkennung und Sicherheitsabgrenzung – 10 Punkte

Sehr gut: Red Flags sind szenariospezifisch, getestet, nachvollziehbar protokolliert und führen zu klarer Eskalation.

Gut: Red Flags sind weitgehend sinnvoll umgesetzt, kleinere Lücken in Begründung oder Tests.

Ausreichend: einfache Red-Flag-Erkennung vorhanden, aber unvollständig oder schwach getestet.

Mangelhaft: Red Flags fehlen, werden ignoriert oder das System gibt unzulässige Entwarnungen.

Datenschutz und ethische Reflexion – 7 Punkte

Sehr gut: konsequente synthetische Daten, Datensparsamkeit, klare Speicherkonzepte, transparente Grenzen, ethische Reflexion sowie nachvollziehbare Einordnung, warum der Prototyp kein einsatzfähiges Medizinprodukt ist.

Gut: Datenschutzerfordernungen überwiegend erfüllt, kleinere Dokumentationslücken.

Ausreichend: grundlegende Datenschutzmaßnahmen vorhanden, aber wenig systematisch.

Mangelhaft: unsachgemäßer Umgang mit Daten oder fehlende Abgrenzung medizinischer Verantwortung.

Testautomatisierung und Softwarequalität – 10 Punkte

Sehr gut: risikoorientierte Tests, gute Abdeckung der Kernlogik, Integrationstests, Fehlerfalltests und nachvollziehbarer Testbericht.

Gut: solide Tests für wichtige Komponenten, kleinere Lücken bei Sonderfällen.

Ausreichend: grundlegende Tests vorhanden, aber geringe Tiefe oder Abdeckung.

Mangelhaft: kaum automatisierte Tests oder Tests prüfen nicht die sicherheitsrelevanten Anforderungen.

Benutzeroberfläche und Avatar-Interaktion – 6 Punkte

Sehr gut: klare, robuste UI mit hilfreichem Avatar, konsistenten Zuständen und guter Bedienbarkeit.

Gut: funktionale UI mit Avatar und kleineren Bedienungsproblemen.

Ausreichend: einfache UI vorhanden, Avatar nur rudimentär.

Mangelhaft: keine brauchbare grafische Ausgabe oder keine erkennbare Avatar-Komponente.

Einsatz und Reflexion von KI-Entwicklungstools – 7 Punkte

Sehr gut: KI-Einsatz ist systematisch dokumentiert, Prompts sind nachvollziehbar, Ergebnisse wurden kritisch geprüft und Risiken reflektiert.

Gut: KI-Einsatz gut dokumentiert, Reflexion mit kleineren Lücken.

Ausreichend: KI-Einsatz erwähnt, aber wenig analysiert.

Mangelhaft: KI-Einsatz verschleiert, unkritisch übernommen oder nicht dokumentiert.

Dokumentation – 7 Punkte

Sehr gut: vollständig, klar strukturiert, fachlich sauber, mit Architektur, Tests, Quellen, Installation und Reflexion.

Gut: weitgehend vollständig und verständlich, kleinere Lücken.

Ausreichend: wesentliche Inhalte vorhanden, aber knapp oder uneinheitlich.

Mangelhaft: Dokumentation fehlt weitgehend oder erlaubt keine Bewertung.

Präsentation und Demo – 4 Punkte

Sehr gut: überzeugende, stabile Demo mit klarer Rollenverteilung, realistischen Testfällen und reflektierter Einordnung.

Gut: verständliche Demo mit kleineren technischen oder didaktischen Schwächen.

Ausreichend: Demo zeigt Grundfunktionalität, aber wenig strukturiert.

Mangelhaft: Demo nicht lauffähig oder fachlich nicht nachvollziehbar.

Teamarbeit und Projektmanagement – 3 Punkte

Sehr gut: klare Rollen, regelmäßige Abstimmung, nachvollziehbare Aufgabenverteilung und professioneller Umgang mit Problemen.

Gut: überwiegend gute Zusammenarbeit mit kleineren Koordinationsproblemen.

Ausreichend: Zusammenarbeit erkennbar, aber wenig strukturiert.

Mangelhaft: unklare Beiträge, fehlende Koordination oder nicht nachvollziehbare Teamleistung.

18. Mindestanforderungen für das Bestehen

Zum Bestehen müssen alle folgenden Punkte erfüllt sein:

- lauffähiger Prototyp,
- JSON-Import synthetischer Patientendaten über URL oder lokal simulierbare URL,
- Identitätsprüfung über Name und Geburtsdatum,
- mindestens drei vollständig implementierte Szenarien,
- strukturierter Dialog mit Start, Zustimmung, Anamnese, Zusammenfassung und Abschluss,

- strukturierte Ergebniszusammenfassung für ärztliches Personal,
- mindestens ein Gerätesimulator oder eine echte Geräteschnittstelle,
- Red-Flag-Erkennung mit Eskalationshinweis,
- grundlegende automatisierte Tests,
- Dokumentation,
- Abschlussdemo,
- klare Kennzeichnung als Assistenzsystem ohne Diagnose- oder Therapieentscheidung.

Nicht bestanden ist das Projekt insbesondere dann, wenn das System medizinische Entwarnungen ausspricht, Diagnosen stellt, Therapieempfehlungen gibt, reale Patientendaten ohne Konzept verarbeitet oder kein lauffähiger Prototyp demonstriert werden kann.

19. Optionale Erweiterungen für leistungsstarke Teams

Mögliche Erweiterungen sind:

- fünftes medizinisches Szenario,
- echte Bluetooth-Integration,
- LangGraph-basierte Agentenarchitektur,
- lokale Sprachverarbeitung,
- verbesserte Avatar-Animation,
- mehrsprachige Patientenschnittstelle,
- Export als FHIR-kompatible Struktur,
- Rollenmodell für Praxispersonal,
- Audit-Log mit differenzierten Ereignistypen,
- Dockerisierung,
- CI-Pipeline,
- erweiterte Usability-Studie,
- Barrierefreiheitsverbesserungen,
- Admin-Oberfläche für Szenario- und Geräte-Konfiguration.

Optionale Erweiterungen werden nur positiv bewertet, wenn die Mindestanforderungen stabil erfüllt sind.

20. Hinweise für Studierende zum professionellen Einsatz von Codex und anderen KI-Tools

20.1 KI als Entwicklungsunterstützung, nicht als Autorität

Codex und vergleichbare Werkzeuge können die Entwicklung beschleunigen, ersetzen aber nicht Ihr Verständnis. Sie bleiben verantwortlich für Architektur, Code, Tests, medizinische Grenzen, Datenschutz und Dokumentation.

20.2 Kleine Aufgaben statt großer Komplettaufträge

Zerlegen Sie Aufgaben in kleine Einheiten:

- Datenmodell für Patientenliste,
- JSON-Validierung,
- Dialogzustand *Identitätsprüfung*,
- Red-Flag-Regel für Brustschmerz,
- Gerätesimulator für Blutdruck,
- Exportfunktion für Zusammenfassung,
- Unit-Tests für Konfigurationsfehler.

Jede Einheit soll einzeln testbar sein.

20.3 Beispiel-Prompt für eine Implementierungsaufgabe

Wir entwickeln einen KI-gestützten Anamnese-Agenten als Hochschulprojekt. Implementiere eine reine Python-Komponente `RedFlagEngine` ohne UI-Abhängigkeiten. Die Engine erhält ein strukturiertes Dictionary für das Szenario `chest_pain` und gibt eine Liste ausgelöster Red Flags zurück. Keine Diagnose, keine Therapieempfehlung, keine Entwarnung. Schreibe pytest-Tests für Brustschmerz mit Atemnot, Ausstrahlung in den linken Arm, Synkope, unauffälligen Routinefall und fehlende Felder. Dokumentiere Annahmen im Code.

20.4 Beispiel-Prompt für Code Review

Prüfe den folgenden Code auf Testbarkeit, Fehlerbehandlung, Datenschutz, medizinisch riskante Ausgaben und Verletzungen der Anforderung, keine Diagnose oder Therapieempfehlung zu geben. Schlage konkrete Verbesserungen vor, aber ändere keine öffentlichen Schnittstellen ohne Begründung.

20.5 Beispiel-Prompt für Tests

Erstelle zusätzliche Unit-Tests für die Funktion `build_summary`. Die Tests sollen prüfen, dass Red Flags separat ausgewiesen werden, simulierte Messwerte als simuliert markiert sind, keine Diagnose ausgegeben wird und fehlende optionale Felder die Ausgabe nicht abbrechen lassen.

20.6 Verbotene oder riskante KI-Nutzung

Nicht zulässig sind:

- ungeprüfte Übernahme medizinischer Regeln aus KI-Ausgaben,
- Verwendung realer Patientendaten in Prompts,
- Generierung von Diagnose- oder Therapieaussagen,
- Einbau nicht verstandener Bibliotheken,
- Abgabe von Code, den das Team nicht erklären kann,
- Verschweigen wesentlicher KI-generierter Bestandteile.

20.7 Erwartete Reflexion

Jedes Team dokumentiert mindestens:

- welche KI-Tools genutzt wurden,
- für welche Aufgaben sie genutzt wurden,
- welche Prompts besonders hilfreich waren,
- welche Vorschläge fehlerhaft oder riskant waren,
- wie KI-generierter Code geprüft wurde,
- welche Tests durch KI-Unterstützung entstanden sind,
- welche Grenzen des KI-Einsatzes sichtbar wurden.

20.8 Abschlussgedanke

Das Ziel des Projekts ist nicht, ein marktreifes Medizinprodukt zu entwickeln. Ziel ist ein professionell entwickelter, kritisch reflektierter und sicher abgegrenzter Prototyp, an dem Sie lernen, wie KI-gestützte Softwareentwicklung in einem anspruchsvollen, medizinisch sensiblen Kontext verantwortungsvoll eingesetzt werden kann.